

Compulsory Assignment 2: Convolutional neural networks

Please fill out the group name, number, members and optionally the name below.

Group number: 26

Group member 1: Milad Tootoonchi

Group member 2: Sania Minaipour

Group member 3: Makka Dulgaeva

Assignment Submission

To complete this assignment answer the relevant questions in this notebook and write the code required to implement the relevant models. The assignment is submitted by handing in this notebook as an .ipynb file and as a .pdf file.

Introduction

In this assignment, you will build a Convolutional Neural Network (CNN) to classify images of natural scenes from around the world. Dataset: Intel-image-classification

<https://www.kaggle.com/datasets/puneet6060/intel-image-classification>

This Data contains around 25k images of size 150x150 distributed under 6 categories.

{'buildings' -> 0, 'forest' -> 1, 'glacier' -> 2, 'mountain' -> 3, 'sea' -> 4, 'street' -> 5 }

In CA2, we use only 3 classes "buildings", "forest", "sea".

This data was initially published on <https://datahack.analyticsvidhya.com> by Intel to host a Image classification Challenge.

Landscape Pictures

Example image:



Assignment structure

1. Part 0: Setup & Data
2. Part 1: Baseline CNN (Clean Data)
3. Part 2: Choose Your Robustness Challenges
4. Part 3: Results & Comparison

```
## Library imports
```

```
In [70]: # Feel free to add or remove libraries as you want
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import seaborn as sns
import tensorflow.keras as ks
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import tensorflow as tf
from tensorflow.keras import layers, models
from utilities import f1_score, resize_dataset, plot_training_history, add_gaussian
```

Part 0: Setup & Data, CNN for Landscape Picture dataset

NOTE FOR STUDENTS:

Do not forget to change the path of root_dir below so it points to the Intel folder which contains the dataset inside your CA2 directory.

Example:

Windows: r"C:\Users\yourname...\CA2\Intel"

Mac/Linux: "/home/yourname/CA2/Intel"

Loading DATASET

```
In [74]: import importlib
import utilities

# Force reload the updated utilities.py
importlib.reload(utilities)

from utilities import load_intel_dataset
```

```
In [75]: # Load dataset
data = load_intel_dataset(
    root_dir=r"C:\Users\milad\Documents\Proggng\Machine-Learning-Deep-Learning\DAT
    img_size=(224, 224),
    selected_classes=["buildings", "forest", "sea"], # only 3 classes
    verbose=1

)

X_train, y_train = data["X_train"], data["y_train"]
X_val, y_val = data["X_val"], data["y_val"]
X_test, y_test = data["X_test"], data["y_test"]
```

```

class_names      = data["class_names"]

print("Train:", X_train.shape, y_train.shape)
print("Val:",   X_val.shape,   y_val.shape)
print("Test:",  X_test.shape,  y_test.shape)

# Show number of images per class
def count_per_class(y, split_name):
    unique, counts = np.unique(y, return_counts=True)
    print(f"\n{split_name} set class distribution:")
    for cls, cnt in zip(unique, counts):
        print(f" {class_names[cls]}: {cnt}")

count_per_class(y_train, "Train")
count_per_class(y_val,   "Val")
count_per_class(y_test,  "Test")

```

```

[train] Loaded 500/6736 images
[train] Loaded 1000/6736 images
[train] Loaded 1500/6736 images
[train] Loaded 2000/6736 images
[train] Loaded 2500/6736 images
[train] Loaded 3000/6736 images
[train] Loaded 3500/6736 images
[train] Loaded 4000/6736 images
[train] Loaded 4500/6736 images
[train] Loaded 5000/6736 images
[train] Loaded 5500/6736 images
[train] Loaded 6000/6736 images
[train] Loaded 6500/6736 images
[train] Loaded 6736/6736 images
[test] Loaded 500/1421 images
[test] Loaded 1000/1421 images
[test] Loaded 1421/1421 images
Final shapes → Train: (6062, 224, 224, 3), Val: (674, 224, 224, 3), Test: (1421, 224, 224, 3)
Classes: ['buildings', 'forest', 'sea']
Train: (6062, 224, 224, 3) (6062,)
Val: (674, 224, 224, 3) (674,)
Test: (1421, 224, 224, 3) (1421,)

Train set class distribution:
  buildings: 1972
  forest: 2044
  sea: 2046

Val set class distribution:
  buildings: 219
  forest: 227
  sea: 228

Test set class distribution:
  buildings: 437
  forest: 474
  sea: 510

```


Preprocessing

```
In [76]: # Normalizing input between [0,1]
X_train = X_train.astype("float32") / 255.0 if X_train.size else X_train
X_val    = X_val.astype("float32")    / 255.0 if X_val.size else X_val
X_test   = X_test.astype("float32")   / 255.0 if X_test.size else X_test

# Converting targets from numbers to categorical format
if y_train is not None and y_train.size:
    num_classes = len(np.unique(y_train))
    y_train = ks.utils.to_categorical(y_train, num_classes)

if y_val is not None and y_val.size:
    y_val = ks.utils.to_categorical(y_val, num_classes)

if y_test is not None and y_test.size:
    y_test = ks.utils.to_categorical(y_test, num_classes)

print("X_train shape:", X_train.shape)
print("X_val shape:", X_val.shape)
print("X_test shape:", X_test.shape)
```

X_train shape: (6062, 224, 224, 3)

X_val shape: (674, 224, 224, 3)

X_test shape: (1421, 224, 224, 3)

Part 1: Build a CNN network with the LeNet5 architecture

Implement LeNet5 architecture for Landscape Pictures (RGB):

The LeNet architecture takes a $32 \times 32 \times C$ image as input, where C is the number of color channels. You may use `resize_dataset()` and `f1_score()` from `utilities.py`

Input & resizing: Resize all images to 32×32 and use $C = 3$ channels (RGB). If you choose a different input size, update the intermediate shapes accordingly, but keep the LeNet-5 pattern (Conv → Pool → Conv → Pool → FC → FC → FC).

Layer 1 - Convolution (5x5): The output shape should be $28 \times 28 \times 6$. **Activation:** ReLU.

MaxPooling: The output shape should be $14 \times 14 \times 6$.

Layer 2 - Convolution (5x5): The output shape should be $10 \times 10 \times 16$. **Activation:** ReLU.

MaxPooling: The output shape should be $5 \times 5 \times 16$.

Flatten: Flatten the output shape of the final pooling layer such that it's 1D instead of 3D. You may need to use `tf.reshape`.

Layer 3 - Fully Connected: This should have 120 outputs. **Activation:** ReLU.

Layer 4 - Fully Connected: This should have 84 outputs. **Activation:** ReLU.

Layer 5 - Fully Connected (output): `num_classes` . **Activation:** softmax

Compile the network with the

- `tf.keras.losses.CategoricalCrossentropy` loss function
- the `adam` optimizer
- with the `accuracy` metric and (your own implementation of the) F1-score metric.

```
In [50]: def f1(y_true, y_pred):
    y_pred = tf.cast(tf.argmax(y_pred, axis = -1), tf.int32)
    y_true = tf.cast(tf.argmax(y_true, axis = -1), tf.int32)

    # Convert to one-hot for matching shapes
    y_pred = tf.one_hot(y_pred, depth = tf.shape(y_true)[-1])
    y_true = tf.one_hot(y_true, depth = tf.shape(y_true)[-1])

    tp = tf.reduce_sum(tf.cast(y_true * y_pred, tf.float32))
    fp = tf.reduce_sum(tf.cast((1 - y_true) * y_pred, tf.float32))
    fn = tf.reduce_sum(tf.cast(y_true * (1 - y_pred), tf.float32))

    precision = tp / (tp + fp + 1e-8)
    recall = tp / (tp + fn + 1e-8)
    f1 = 2 * precision * recall / (precision + recall + 1e-8)
    return f1
```

```
In [51]: X_train_resized = resize_dataset(X_train)
X_test_resized = resize_dataset(X_test)
X_val_resized = resize_dataset(X_val)

def lanet():
    inputs = layers.Input(shape = (32, 32, 3))

    x = layers.Conv2D(6, kernel_size = (5, 5), padding = 'valid', activation = 'relu')
    x = layers.MaxPool2D(pool_size = (2, 2), strides = 2)(x)

    x = layers.Conv2D(16, kernel_size = (5, 5), padding = 'valid', activation = 'relu')
    x = layers.MaxPool2D(pool_size = (2, 2), strides = 2)(x)

    # Flattening
    x = layers.Flatten()(x)

    x = layers.Dense(120, activation = 'relu')(x)
    x = layers.Dense(84, activation = 'relu')(x)
    x = layers.Dense(num_classes, activation = 'softmax')(x)

    model = models.Model(inputs = inputs, outputs = x)

    model.compile(
        optimizer = ks.optimizers.Adam(),
```

```
        loss = ks.losses.CategoricalCrossentropy(),  
        metrics = ["accuracy", f1]  
    )  
  
    return model
```

Task 1.1.2 Train network

Train the network with a

- batch size of 64 samples
- for 20 epochs

```
In [52]: model = lanet()  
  
history = model.fit(  
    X_train_resized, y_train,  
    batch_size = 64,  
    epochs = 20,  
    validation_data = (X_val_resized, y_val),  
    verbose = 1  
)
```

Epoch 1/20
95/95 ————— 3s 12ms/step - accuracy: 0.4812 - f1: 0.4814 - loss: 0.99
17 - val_accuracy: 0.6365 - val_f1: 0.6332 - val_loss: 0.7669

Epoch 2/20
95/95 ————— 1s 8ms/step - accuracy: 0.6041 - f1: 0.6040 - loss: 0.822
7 - val_accuracy: 0.6395 - val_f1: 0.6410 - val_loss: 0.7475

Epoch 3/20
95/95 ————— 1s 8ms/step - accuracy: 0.6255 - f1: 0.6255 - loss: 0.792
9 - val_accuracy: 0.6380 - val_f1: 0.6384 - val_loss: 0.7458

Epoch 4/20
95/95 ————— 1s 8ms/step - accuracy: 0.6217 - f1: 0.6218 - loss: 0.787
1 - val_accuracy: 0.6499 - val_f1: 0.6485 - val_loss: 0.7244

Epoch 5/20
95/95 ————— 1s 8ms/step - accuracy: 0.6445 - f1: 0.6449 - loss: 0.768
5 - val_accuracy: 0.6617 - val_f1: 0.6611 - val_loss: 0.7168

Epoch 6/20
95/95 ————— 1s 8ms/step - accuracy: 0.6602 - f1: 0.6603 - loss: 0.751
3 - val_accuracy: 0.6691 - val_f1: 0.6682 - val_loss: 0.6929

Epoch 7/20
95/95 ————— 1s 8ms/step - accuracy: 0.6702 - f1: 0.6703 - loss: 0.740
2 - val_accuracy: 0.6751 - val_f1: 0.6751 - val_loss: 0.7134

Epoch 8/20
95/95 ————— 1s 8ms/step - accuracy: 0.6767 - f1: 0.6767 - loss: 0.719
9 - val_accuracy: 0.7047 - val_f1: 0.7060 - val_loss: 0.6654

Epoch 9/20
95/95 ————— 1s 8ms/step - accuracy: 0.6933 - f1: 0.6933 - loss: 0.698
0 - val_accuracy: 0.7122 - val_f1: 0.7157 - val_loss: 0.6505

Epoch 10/20
95/95 ————— 1s 7ms/step - accuracy: 0.7036 - f1: 0.7036 - loss: 0.676
7 - val_accuracy: 0.7122 - val_f1: 0.7119 - val_loss: 0.6333

Epoch 11/20
95/95 ————— 1s 9ms/step - accuracy: 0.7182 - f1: 0.7184 - loss: 0.647
6 - val_accuracy: 0.7671 - val_f1: 0.7695 - val_loss: 0.5782

Epoch 12/20
95/95 ————— 1s 9ms/step - accuracy: 0.7450 - f1: 0.7450 - loss: 0.620
2 - val_accuracy: 0.7315 - val_f1: 0.7316 - val_loss: 0.5877

Epoch 13/20
95/95 ————— 1s 7ms/step - accuracy: 0.7601 - f1: 0.7605 - loss: 0.577
3 - val_accuracy: 0.7923 - val_f1: 0.7949 - val_loss: 0.5140

Epoch 14/20
95/95 ————— 1s 7ms/step - accuracy: 0.7771 - f1: 0.7773 - loss: 0.548
4 - val_accuracy: 0.8205 - val_f1: 0.8206 - val_loss: 0.4763

Epoch 15/20
95/95 ————— 1s 7ms/step - accuracy: 0.7933 - f1: 0.7933 - loss: 0.518
5 - val_accuracy: 0.7760 - val_f1: 0.7767 - val_loss: 0.5421

Epoch 16/20
95/95 ————— 1s 8ms/step - accuracy: 0.7959 - f1: 0.7958 - loss: 0.512
8 - val_accuracy: 0.8338 - val_f1: 0.8334 - val_loss: 0.4522

Epoch 17/20
95/95 ————— 1s 7ms/step - accuracy: 0.8131 - f1: 0.8133 - loss: 0.477
6 - val_accuracy: 0.8175 - val_f1: 0.8178 - val_loss: 0.4623

Epoch 18/20
95/95 ————— 1s 7ms/step - accuracy: 0.8121 - f1: 0.8121 - loss: 0.469
4 - val_accuracy: 0.8383 - val_f1: 0.8402 - val_loss: 0.4364

Epoch 19/20
95/95 ————— 1s 8ms/step - accuracy: 0.8251 - f1: 0.8251 - loss: 0.450

1 - val_accuracy: 0.8487 - val_f1: 0.8488 - val_loss: 0.4168
 Epoch 20/20
 95/95  1s 8ms/step - accuracy: 0.8246 - f1: 0.8248 - loss: 0.453
 9 - val_accuracy: 0.8234 - val_f1: 0.8247 - val_loss: 0.4571

Task 1.1.3

Experiment with different architectures of your choice. Vary the number of filters, try different kernel sizes, add more layers, dropout, early stopping, and modify the fully connected layers. Report the best performance you achieve.

```
In [53]: # SqueezeNet Architecture
def fire_module(x, squeeze_filters, expand_filters):
    # Squeeze Layer (1x1)
    squeeze = layers.Conv2D(squeeze_filters, (1, 1), activation = 'relu', padding =

    # Expand Layer (1x1 and 3x3)
    expand_1x1 = layers.Conv2D(expand_filters, (1, 1), activation = 'relu', padding
    expand_3x3 = layers.Conv2D(expand_filters, (3, 3), activation = 'relu', padding

    # Concatenate
    output = layers.Concatenate(axis = -1)([expand_1x1, expand_3x3])
    return output

def squeezenet(num_classes, size):
    inputs = layers.Input(shape = (size, size, 3))

    x = layers.Conv2D(96, (7, 7), strides = (2, 2), activation = 'relu', padding =
    x = layers.MaxPooling2D((3, 3), strides = (2, 2), padding = 'same')(x)

    x = fire_module(x, 16, 64)
    x = fire_module(x, 16, 64)
    x = fire_module(x, 32, 128)
    x = layers.MaxPooling2D((3, 3), strides = (2, 2), padding = 'same')(x)

    x = fire_module(x, 32, 128)
    x = fire_module(x, 48, 192)
    x = fire_module(x, 48, 192)
    x = fire_module(x, 64, 256)
    x = layers.MaxPooling2D((3, 3), strides = (2, 2), padding = 'same')(x)

    x = fire_module(x, 64, 256)
    x = layers.Dropout(0.5)(x)
    x = layers.Conv2D(num_classes, (1, 1), padding = 'valid')(x)
    x = layers.GlobalAveragePooling2D()(x)
    outputs = layers.Activation('softmax')(x)

    model = models.Model(inputs, outputs)

    model.compile(optimizer = ks.optimizers.Adam(),
                  loss = ks.losses.CategoricalCrossentropy(),
```

```
        metrics = ['accuracy', f1])  
    return model
```

```
In [54]: # Training  
size = 32  
X_train_resized = resize_dataset(X_train)  
X_test_resized = resize_dataset(X_test)  
X_val_resized = resize_dataset(X_val)  
model = squeezenet(num_classes, size)  
  
history_squeeze = model.fit(  
    X_train_resized, y_train,  
    batch_size = 64,  
    epochs = 20,  
    validation_data = (X_val_resized, y_val),  
    verbose = 1  
)  
  
print(f"The val_accuracy score: {history_squeeze.history['val_accuracy'][-1]}")  
print(f"The val_f1 score: {history_squeeze.history['val_f1'][-1]}")
```

Epoch 1/20
95/95  16s 47ms/step - accuracy: 0.3261 - f1: 0.3261 - loss: 1.0991 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0986
Epoch 2/20
95/95  4s 39ms/step - accuracy: 0.3286 - f1: 0.3287 - loss: 1.0986 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985
Epoch 3/20
95/95  4s 39ms/step - accuracy: 0.3311 - f1: 0.3312 - loss: 1.0986 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985
Epoch 4/20
95/95  4s 39ms/step - accuracy: 0.3342 - f1: 0.3340 - loss: 1.0987 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985
Epoch 5/20
95/95  4s 39ms/step - accuracy: 0.3345 - f1: 0.3348 - loss: 1.0987 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985
Epoch 6/20
95/95  4s 40ms/step - accuracy: 0.3360 - f1: 0.3363 - loss: 1.0986 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985
Epoch 7/20
95/95  4s 41ms/step - accuracy: 0.3405 - f1: 0.3404 - loss: 1.0988 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985
Epoch 8/20
95/95  4s 40ms/step - accuracy: 0.3355 - f1: 0.3354 - loss: 1.0986 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985
Epoch 9/20
95/95  4s 43ms/step - accuracy: 0.3354 - f1: 0.3353 - loss: 1.0986 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985
Epoch 10/20
95/95  4s 40ms/step - accuracy: 0.3311 - f1: 0.3312 - loss: 1.0986 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985
Epoch 11/20
95/95  4s 39ms/step - accuracy: 0.3372 - f1: 0.3370 - loss: 1.0986 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985
Epoch 12/20
95/95  4s 41ms/step - accuracy: 0.3242 - f1: 0.3240 - loss: 1.0986 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985
Epoch 13/20
95/95  4s 42ms/step - accuracy: 0.3266 - f1: 0.3264 - loss: 1.0986 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985
Epoch 14/20
95/95  4s 40ms/step - accuracy: 0.3383 - f1: 0.3384 - loss: 1.0986 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985
Epoch 15/20
95/95  4s 43ms/step - accuracy: 0.3357 - f1: 0.3356 - loss: 1.0985 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985
Epoch 16/20
95/95  4s 43ms/step - accuracy: 0.3355 - f1: 0.3354 - loss: 1.0986 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985
Epoch 17/20
95/95  4s 46ms/step - accuracy: 0.3392 - f1: 0.3394 - loss: 1.0985 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985
Epoch 18/20
95/95  4s 44ms/step - accuracy: 0.3359 - f1: 0.3360 - loss: 1.0986 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985
Epoch 19/20
95/95  4s 44ms/step - accuracy: 0.3347 - f1: 0.3343 - loss: 1.09

85 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985

Epoch 20/20

95/95 ————— 4s 42ms/step - accuracy: 0.3327 - f1: 0.3326 - loss: 1.09

85 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985

The val_accuracy score: 0.3382789194583893

The val_f1 score: 0.34141042828559875

```
In [55]: # Inception model
def simple_inception_block(x, f1 = 32, f3 = 32, f5 = 16, pool_proj = 16):
    # 1x1
    path1 = layers.Conv2D(f1, (1,1), padding = 'same', activation = 'relu')(x)

    # 3x3
    path2 = layers.Conv2D(f3, (3,3), padding = 'same', activation = 'relu')(x)

    # 5x5
    path3 = layers.Conv2D(f5, (5,5), padding = 'same', activation = 'relu')(x)

    # 3x3 max pooling + 1x1 conv
    path4 = layers.MaxPooling2D((3,3), strides = (1,1), padding = 'same')(x)
    path4 = layers.Conv2D(pool_proj, (1,1), padding = 'same', activation = 'relu')(x)

    # Concatenate along channels
    return layers.concatenate([path1, path2, path3, path4], axis = -1)

def inception(num_classes, size):
    inputs = layers.Input(shape = (size, size, 3))

    # Stem
    x = layers.Conv2D(32, (3,3), strides = (2,2), padding = 'same', activation = 'relu')(inputs)
    x = layers.Conv2D(32, (3,3), padding = 'same', activation = 'relu')(x)
    x = layers.MaxPooling2D((3,3), strides = (2,2), padding = 'same')(x)

    # Inception blocks
    x = simple_inception_block(x, 32, 32, 16, 16)
    x = simple_inception_block(x, 64, 48, 16, 16)
    x = layers.MaxPooling2D((3,3), strides = (2,2), padding = 'same')(x)

    x = simple_inception_block(x, 64, 64, 32, 32)
    x = simple_inception_block(x, 64, 64, 32, 32)
    x = layers.GlobalAveragePooling2D()(x)

    # Classifier
    x = layers.Dropout(0.5)(x)
    outputs = layers.Dense(num_classes, activation = 'softmax')(x)

    model = models.Model(inputs, outputs)
    model.compile(
        optimizer = ks.optimizers.Adam(),
        loss = ks.losses.CategoricalCrossentropy(),
        metrics = ['accuracy', f1]
    )

    return model
```



```
In [56]: # Training
size = 32
X_train_resized = resize_dataset(X_train)
X_test_resized = resize_dataset(X_test)
X_val_resized = resize_dataset(X_val)
model = inception(num_classes, size)

history_inception = model.fit(
    X_train_resized, y_train,
    batch_size = 64,
    epochs = 20,
    validation_data = (X_val_resized, y_val),
    verbose = 1
)

print(f"The val_accuracy score: {history_inception.history['val_accuracy'][-1]}")
print(f"The val_f1 score: {history_inception.history['val_f1'][-1]}")
```

Epoch 1/20
95/95  11s 45ms/step - accuracy: 0.3311 - f1: 0.3312 - loss: 1.0987 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985

Epoch 2/20
95/95  3s 35ms/step - accuracy: 0.3345 - f1: 0.3348 - loss: 1.0986 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985

Epoch 3/20
95/95  3s 34ms/step - accuracy: 0.3378 - f1: 0.3378 - loss: 1.0986 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985

Epoch 4/20
95/95  3s 33ms/step - accuracy: 0.3428 - f1: 0.3425 - loss: 1.0985 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985

Epoch 5/20
95/95  3s 33ms/step - accuracy: 0.3373 - f1: 0.3376 - loss: 1.0986 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985

Epoch 6/20
95/95  3s 34ms/step - accuracy: 0.3274 - f1: 0.3273 - loss: 1.0987 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985

Epoch 7/20
95/95  3s 33ms/step - accuracy: 0.3301 - f1: 0.3303 - loss: 1.0987 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985

Epoch 8/20
95/95  3s 33ms/step - accuracy: 0.3364 - f1: 0.3365 - loss: 1.0986 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985

Epoch 9/20
95/95  3s 34ms/step - accuracy: 0.3458 - f1: 0.3459 - loss: 1.0985 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985

Epoch 10/20
95/95  3s 33ms/step - accuracy: 0.3375 - f1: 0.3371 - loss: 1.0987 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985

Epoch 11/20
95/95  3s 34ms/step - accuracy: 0.3403 - f1: 0.3405 - loss: 1.0986 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985

Epoch 12/20
95/95  3s 34ms/step - accuracy: 0.3345 - f1: 0.3347 - loss: 1.0985 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985

Epoch 13/20
95/95  3s 34ms/step - accuracy: 0.3332 - f1: 0.3329 - loss: 1.0985 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985

Epoch 14/20
95/95  3s 35ms/step - accuracy: 0.3284 - f1: 0.3285 - loss: 1.0988 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985

Epoch 15/20
95/95  3s 35ms/step - accuracy: 0.3339 - f1: 0.3336 - loss: 1.0986 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985

Epoch 16/20
95/95  3s 35ms/step - accuracy: 0.3362 - f1: 0.3363 - loss: 1.0985 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985

Epoch 17/20
95/95  3s 34ms/step - accuracy: 0.3398 - f1: 0.3398 - loss: 1.0985 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985

Epoch 18/20
95/95  3s 34ms/step - accuracy: 0.3378 - f1: 0.3378 - loss: 1.0986 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985

Epoch 19/20
95/95  3s 34ms/step - accuracy: 0.3334 - f1: 0.3329 - loss: 1.09

86 - val_accuracy: 0.3368 - val_f1: 0.3350 - val_loss: 1.0985
 Epoch 20/20
 95/95 ————— 3s 34ms/step - accuracy: 0.3373 - f1: 0.3372 - loss: 1.09
 86 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 1.0985
 The val_accuracy score: 0.3382789194583893
 The val_f1 score: 0.34141042828559875

```
In [59]: # ResNet
def residual_block(x, filters, stride = 1):
    shortcut = x

    x = layers.Conv2D(filters, (3, 3), strides=stride, padding = 'same', activation=None)(x)
    x = layers.BatchNormalization()(x)

    x = layers.Conv2D(filters, (3, 3), padding = 'same', activation=None)(x)
    x = layers.BatchNormalization()(x)

    if shortcut.shape[-1] != filters or stride != 1:
        shortcut = layers.Conv2D(filters, (1, 1), strides=stride, padding = 'same', activation=None)(x)
        shortcut = layers.BatchNormalization()(shortcut)

    x = layers.Add()([x, shortcut])
    x = layers.ReLU()(x)
    return x

def resnet(num_classes, size):
    inputs = layers.Input(shape = (size, size, 3))

    x = layers.Conv2D(32, (3, 3), strides=1, padding='same', activation='relu')(inputs)
    x = layers.BatchNormalization()(x)

    x = residual_block(x, 32)
    x = residual_block(x, 32)

    x = residual_block(x, 64, stride=2)
    x = residual_block(x, 64)

    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.5)(x)

    outputs = layers.Dense(num_classes, activation = 'softmax')(x)

    model = models.Model(inputs, outputs)
    model.compile(
        optimizer = ks.optimizers.Adam(),
        loss = ks.losses.CategoricalCrossentropy(),
        metrics = ['accuracy', f1]
    )
    return model
```

```
In [60]: # Training
size = 32
X_train_resized = resize_dataset(X_train)
X_test_resized = resize_dataset(X_test)
X_val_resized = resize_dataset(X_val)
model = resnet(num_classes, size)
```

```
history_resnet = model.fit(  
    X_train_resized, y_train,  
    batch_size = 64,  
    epochs = 20,  
    validation_data = (X_val_resized, y_val),  
    verbose = 1  
)  
  
print(f"The val_accuracy score: {history_resnet.history['val_accuracy'][-1]}")  
print(f"The val_f1 score: {history_resnet.history['val_f1'][-1]}")
```

Epoch 1/20
95/95  36s 267ms/step - accuracy: 0.7842 - f1: 0.7845 - loss: 0.5768 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 2.7073

Epoch 2/20
95/95  23s 244ms/step - accuracy: 0.8669 - f1: 0.8668 - loss: 0.3521 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 5.2614

Epoch 3/20
95/95  24s 248ms/step - accuracy: 0.8837 - f1: 0.8832 - loss: 0.3051 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 5.4028

Epoch 4/20
95/95  40s 235ms/step - accuracy: 0.9099 - f1: 0.9099 - loss: 0.2492 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 2.6898

Epoch 5/20
95/95  20s 208ms/step - accuracy: 0.9200 - f1: 0.9200 - loss: 0.2231 - val_accuracy: 0.3383 - val_f1: 0.3414 - val_loss: 2.9549

Epoch 6/20
95/95  19s 203ms/step - accuracy: 0.9304 - f1: 0.9303 - loss: 0.1967 - val_accuracy: 0.4525 - val_f1: 0.4545 - val_loss: 1.6920

Epoch 7/20
95/95  19s 199ms/step - accuracy: 0.9363 - f1: 0.9361 - loss: 0.1792 - val_accuracy: 0.7136 - val_f1: 0.7171 - val_loss: 0.9414

Epoch 8/20
95/95  18s 191ms/step - accuracy: 0.9414 - f1: 0.9415 - loss: 0.1708 - val_accuracy: 0.7255 - val_f1: 0.7284 - val_loss: 0.8338

Epoch 9/20
95/95  19s 203ms/step - accuracy: 0.9525 - f1: 0.9526 - loss: 0.1443 - val_accuracy: 0.9125 - val_f1: 0.9099 - val_loss: 0.2521

Epoch 10/20
95/95  18s 189ms/step - accuracy: 0.9551 - f1: 0.9552 - loss: 0.1318 - val_accuracy: 0.8828 - val_f1: 0.8828 - val_loss: 0.2888

Epoch 11/20
95/95  18s 194ms/step - accuracy: 0.9597 - f1: 0.9597 - loss: 0.1167 - val_accuracy: 0.8947 - val_f1: 0.8954 - val_loss: 0.2972

Epoch 12/20
95/95  18s 188ms/step - accuracy: 0.9639 - f1: 0.9639 - loss: 0.1078 - val_accuracy: 0.8665 - val_f1: 0.8659 - val_loss: 0.3912

Epoch 13/20
95/95  18s 191ms/step - accuracy: 0.9630 - f1: 0.9631 - loss: 0.1126 - val_accuracy: 0.8961 - val_f1: 0.8943 - val_loss: 0.3069

Epoch 14/20
95/95  19s 201ms/step - accuracy: 0.9723 - f1: 0.9723 - loss: 0.0913 - val_accuracy: 0.9362 - val_f1: 0.9364 - val_loss: 0.2190

Epoch 15/20
95/95  18s 193ms/step - accuracy: 0.9673 - f1: 0.9674 - loss: 0.0976 - val_accuracy: 0.9199 - val_f1: 0.9195 - val_loss: 0.2391

Epoch 16/20
95/95  18s 193ms/step - accuracy: 0.9772 - f1: 0.9772 - loss: 0.0746 - val_accuracy: 0.9466 - val_f1: 0.9476 - val_loss: 0.1728

Epoch 17/20
95/95  19s 195ms/step - accuracy: 0.9772 - f1: 0.9773 - loss: 0.0712 - val_accuracy: 0.8843 - val_f1: 0.8854 - val_loss: 0.3054

Epoch 18/20
95/95  18s 192ms/step - accuracy: 0.9805 - f1: 0.9805 - loss: 0.0647 - val_accuracy: 0.9095 - val_f1: 0.9121 - val_loss: 0.4323

Epoch 19/20
95/95  19s 195ms/step - accuracy: 0.9749 - f1: 0.9749 - loss: 0.

0749 - val_accuracy: 0.8145 - val_f1: 0.8162 - val_loss: 1.0283

Epoch 20/20

95/95 ————— 19s 195ms/step - accuracy: 0.9804 - f1: 0.9803 - loss: 0.

0589 - val_accuracy: 0.9273 - val_f1: 0.9266 - val_loss: 0.2221

The val_accuracy score: 0.9272996783256531

The val_f1 score: 0.9266377091407776

The best performance comes from the ResNet architecture, with both validation accuracy and -f1-score on around 0.927. Even though this model overfitted extremely at first, it got more generalized for each epoch, and still did better than the other models.

Task 1.2 Evaluaiton

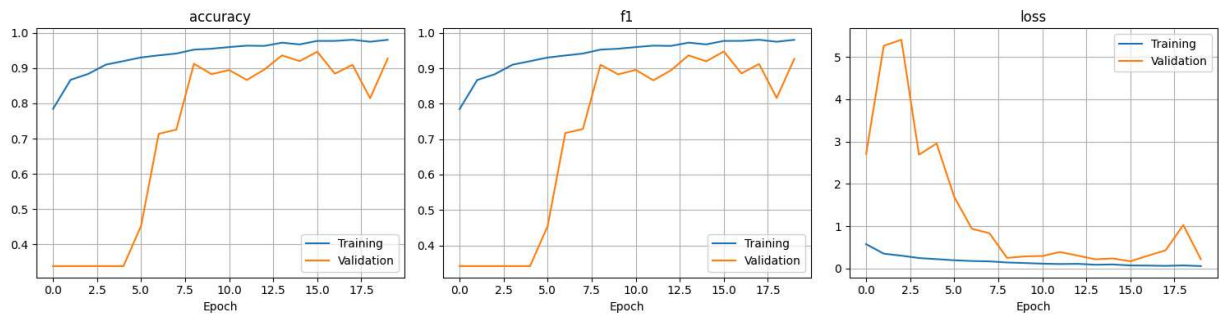
Task 1.2.1 Plot training history

- Plot the training/validation accuracy and loss curves (plot_training_history()) is in utilities.py).
- Report the final validation accuracy (f1_score()) is in utilities.py).

```
In [65]: """ assuming we're only plotting and validating for the best model (resnet)"""
plot_training_history(history_resnet)

y_val_pred_resnet = model.predict(X_val_resized)

val_f1_resnet = f1_score(y_val, y_val_pred_resnet)
print(f"Validation F1 Score (ResNet): {val_f1_resnet:.4f}")
```



22/22 ————— 1s 24ms/step

Validation F1 Score (ResNet): 0.9732

Task 1.2.2 Evaluate on the test dataset

- Report test accuracy (and F1 score (f1_score()) is in utilities.py)).

```
In [66]: y_test_pred_resnet = model.predict(X_test_resized)

test_f1_resnet = f1_score(y_test, y_test_pred_resnet)
print(f"Test F1 Score (ResNet): {test_f1_resnet:.4f}")
```

45/45 ————— 1s 25ms/step

Test F1 Score (ResNet): 0.9787

Task 1.2.3 Create a confusion matrix for both training and testing data

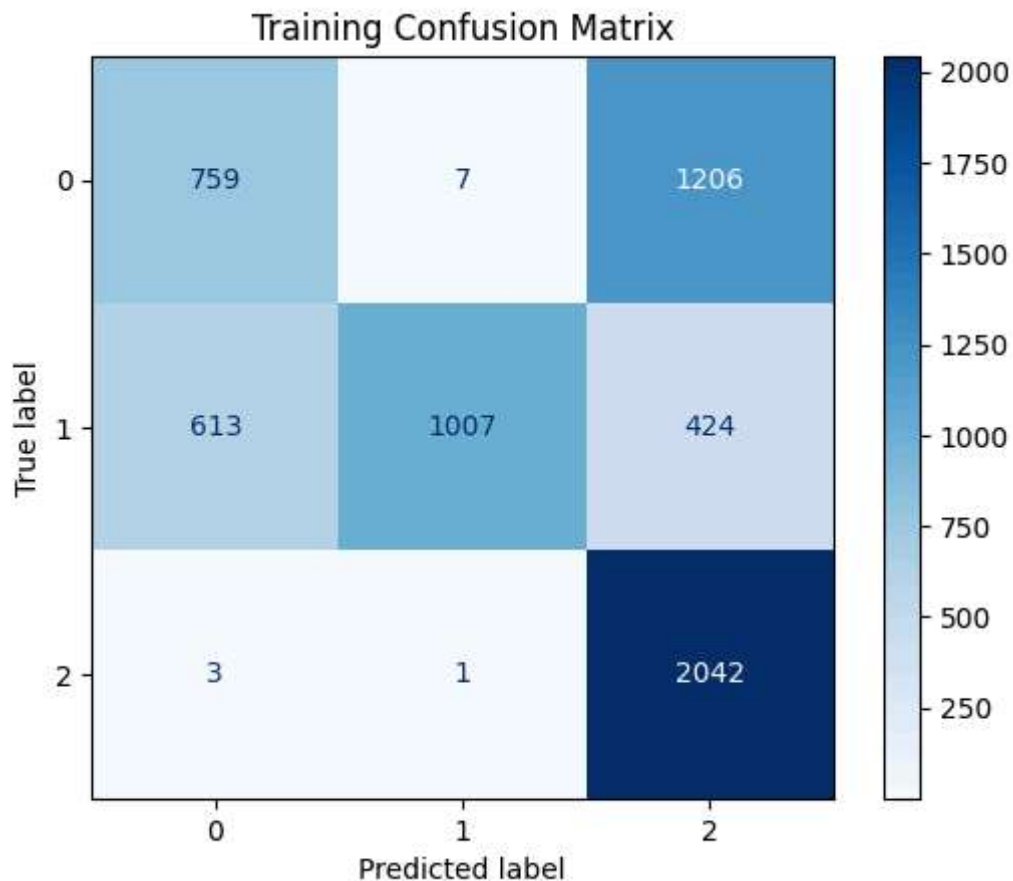
- Visualize confusion matrices.
- Do the test data and train data predict the same items wrong?

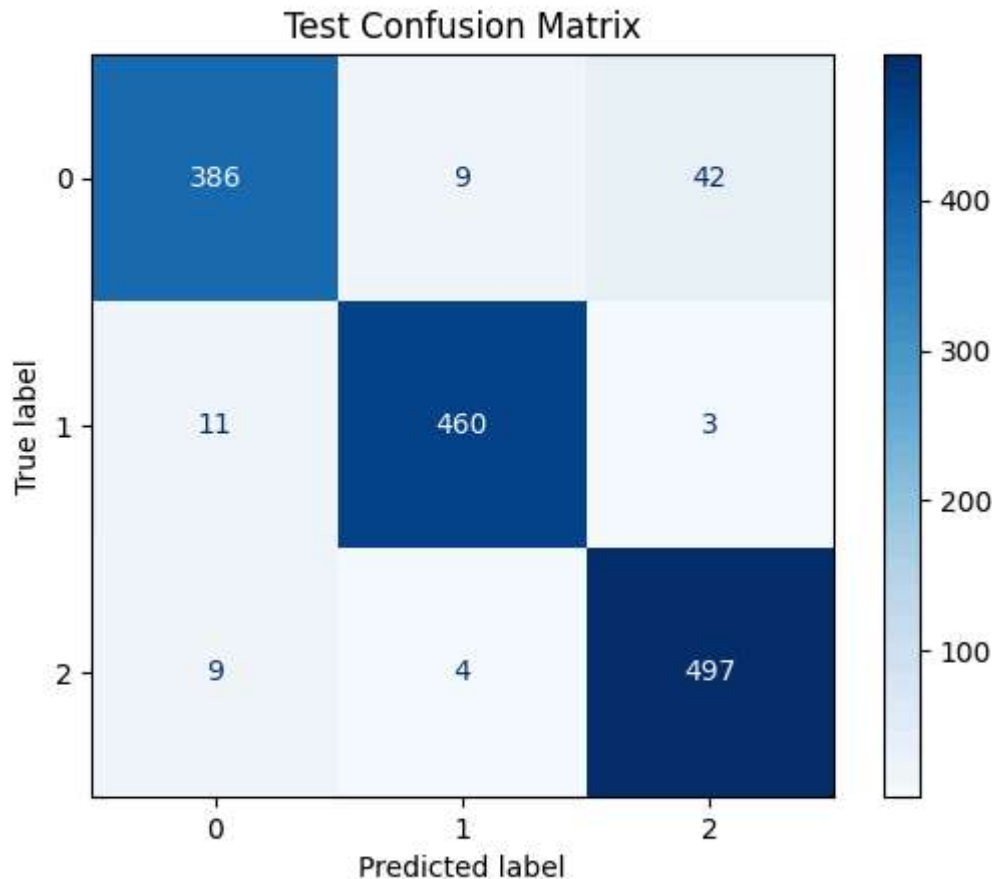
```
In [68]: y_train_pred = model.predict(X_train)

y_train_pred_labels = np.argmax(y_train_pred, axis=1)
y_train_true_labels = np.argmax(y_train, axis=1)
cm_train = confusion_matrix(y_train_true_labels, y_train_pred_labels)
ConfusionMatrixDisplay(cm_train).plot(cmap = 'Blues')
plt.title("Training Confusion Matrix")
plt.show()

y_test_pred_labels = np.argmax(y_test_pred_resnet, axis=1)
y_test_true_labels = np.argmax(y_test, axis=1)
cm_test = confusion_matrix(y_test_true_labels, y_test_pred_labels)
ConfusionMatrixDisplay(cm_test).plot(cmap = 'Blues')
plt.title("Test Confusion Matrix")
plt.show()
```

190/190 ————— 217s 1s/step





The train data predicts more items wrong in general. The train data confuses the sea labels a lot with building and sometimes with forests. And the train data also confuses some buildings with forests. The test data does not confuse as much items like the train data, however that can maybe be a result of the test data being smaller. The test data has the same confusion tendencies as the train data.

Part 2: Robustness (choose two or more), You may use `add_gaussian_noise()` and `add_motion_blur()` from `utilities.py`

Task 2.1 Data Augmentation (training-time)

- Implement an augmentation pipeline.
- Train a model with augmentation and compare against the baseline from Part 1.

Task 2.2 Noise Robustness (test-time corruptions)

- Create corrupted versions of the test images and evaluate the baseline model:
 - Gaussian noise
 - Motion blur
- Report accuracy on the clean test set vs. each corrupted test set.

Task 2.3 Discussion (no coding)

- Which noise types affect the model most?
- What techniques could improve robustness (data augmentation, adversarial training, denoising pre-processing, larger models)?

```
In [71]: # 2.1
datagen = ImageDataGenerator(
    rotation_range = 20,
    width_shift_range = 0.2,
    height_shift_range = 0.2,
    zoom_range = 0.2,
    horizontal_flip = True
)

# Fit to your training data
datagen.fit(X_train_resized)

# Use the augmented data for training
model.fit(
    datagen.flow(X_train_resized, y_train, batch_size = 64),
    epochs = 20,
    validation_data=(X_val, y_val)
)
```

Epoch 1/20

```
c:\Users\milad\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\t
rainers\data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class
should call `super().__init__(**kwargs)` in its constructor. `**kwargs` can include
`workers`, `use_multiprocessing`, `max_queue_size`. Do not pass these arguments to `
fit()`, as they will be ignored.
    self._warn_if_super_not_called()
```

95/95 ————— 48s 473ms/step - accuracy: 0.8865 - f1: 0.8865 - loss: 0.3242 - val_accuracy: 0.5579 - val_f1: 0.5312 - val_loss: 1.5150
Epoch 2/20

95/95 ————— 42s 443ms/step - accuracy: 0.9068 - f1: 0.9070 - loss: 0.2557 - val_accuracy: 0.8294 - val_f1: 0.7912 - val_loss: 0.4708
Epoch 3/20

95/95 ————— 42s 449ms/step - accuracy: 0.9139 - f1: 0.9139 - loss: 0.2355 - val_accuracy: 0.7077 - val_f1: 0.6747 - val_loss: 0.8617
Epoch 4/20

95/95 ————— 43s 451ms/step - accuracy: 0.9233 - f1: 0.9232 - loss: 0.2212 - val_accuracy: 0.7092 - val_f1: 0.6761 - val_loss: 0.6705
Epoch 5/20

95/95 ————— 42s 443ms/step - accuracy: 0.9264 - f1: 0.9263 - loss: 0.2164 - val_accuracy: 0.7760 - val_f1: 0.7401 - val_loss: 0.5803
Epoch 6/20

95/95 ————— 42s 446ms/step - accuracy: 0.9258 - f1: 0.9259 - loss: 0.2054 - val_accuracy: 0.7878 - val_f1: 0.7514 - val_loss: 0.5034
Epoch 7/20

95/95 ————— 43s 453ms/step - accuracy: 0.9317 - f1: 0.9316 - loss: 0.1904 - val_accuracy: 0.7611 - val_f1: 0.7259 - val_loss: 0.5533
Epoch 8/20

95/95 ————— 42s 447ms/step - accuracy: 0.9312 - f1: 0.9312 - loss: 0.2024 - val_accuracy: 0.7656 - val_f1: 0.7301 - val_loss: 0.7614
Epoch 9/20

95/95 ————— 42s 442ms/step - accuracy: 0.9363 - f1: 0.9364 - loss: 0.1809 - val_accuracy: 0.7493 - val_f1: 0.7145 - val_loss: 0.6659
Epoch 10/20

95/95 ————— 42s 448ms/step - accuracy: 0.9330 - f1: 0.9331 - loss: 0.1901 - val_accuracy: 0.5564 - val_f1: 0.5298 - val_loss: 1.1688
Epoch 11/20

95/95 ————— 42s 439ms/step - accuracy: 0.9380 - f1: 0.9380 - loss: 0.1684 - val_accuracy: 0.8872 - val_f1: 0.8466 - val_loss: 0.3050
Epoch 12/20

95/95 ————— 44s 462ms/step - accuracy: 0.9371 - f1: 0.9372 - loss: 0.1773 - val_accuracy: 0.7923 - val_f1: 0.7557 - val_loss: 0.6041
Epoch 13/20

95/95 ————— 43s 451ms/step - accuracy: 0.9358 - f1: 0.9356 - loss: 0.1743 - val_accuracy: 0.7967 - val_f1: 0.7599 - val_loss: 0.5789
Epoch 14/20

95/95 ————— 42s 441ms/step - accuracy: 0.9357 - f1: 0.9357 - loss: 0.1850 - val_accuracy: 0.6766 - val_f1: 0.6449 - val_loss: 0.7532
Epoch 15/20

95/95 ————— 43s 450ms/step - accuracy: 0.9350 - f1: 0.9351 - loss: 0.1793 - val_accuracy: 0.6009 - val_f1: 0.5724 - val_loss: 1.1586
Epoch 16/20

95/95 ————— 42s 449ms/step - accuracy: 0.9350 - f1: 0.9349 - loss: 0.1758 - val_accuracy: 0.6647 - val_f1: 0.6335 - val_loss: 0.7738
Epoch 17/20

95/95 ————— 42s 448ms/step - accuracy: 0.9413 - f1: 0.9413 - loss: 0.1664 - val_accuracy: 0.7270 - val_f1: 0.6932 - val_loss: 0.6897
Epoch 18/20

95/95 ————— 42s 443ms/step - accuracy: 0.9441 - f1: 0.9441 - loss: 0.1573 - val_accuracy: 0.5193 - val_f1: 0.4943 - val_loss: 3.2834
Epoch 19/20

95/95 ————— 42s 441ms/step - accuracy: 0.9358 - f1: 0.9357 - loss: 0.1732 - val_accuracy: 0.7522 - val_f1: 0.7173 - val_loss: 0.6450

Epoch 20/20

95/95 ————— 42s 439ms/step - accuracy: 0.9408 - f1: 0.9409 - loss: 0.1582 - val_accuracy: 0.8353 - val_f1: 0.7969 - val_loss: 0.4214

Out[71]: <keras.src.callbacks.history.History at 0x244f04caf90>

The validation accuracy for this data landed on 0.835 and f1 on around 0.797, this is less than the model we got on part 1 with 0.927 on both accuracy and f1 validation. This model on part 2.1 also overfitted alot more.

In [81]: # 2.2

```
X_test_gaussian = add_gaussian_noise(X_test_resized)
X_test_motion = add_motion_blur(X_test_resized)

_, clean, f1_clean = model.evaluate(X_test_resized, y_test, verbose = 0)
_, gaussian, f1_gaussian = model.evaluate(X_test_gaussian, y_test, verbose = 0)
_, motion, f1_motion = model.evaluate(X_test_motion, y_test, verbose = 0)

print("Scores: accuracy | f1-score")
print(f"Clean: {clean} | {f1_clean}")
print(f"Gaussian Noise: {gaussian} | {f1_gaussian}")
print(f"Motion Blur: {motion} | {f1_motion}")
```

```
Scores: accuracy | f1-score
Clean: 0.935960590839386 | 0.9337606430053711
Gaussian Noise: 0.30752989649772644 | 0.30347222089767456
Motion Blur: 0.4060520827770233 | 0.4138889014720917
```

2.3

The gaussian noise affected the model most. since the model was only trained on clean data, it was not used to noisy images. This made the accuracy and F1 score drop a lot. To make the model better, we could train it with noisy images or use methods to remove the noise before testing. Or we could use the data augmentation instead since the scores did not drop as much as gaussian noise or motion blur.

Part 3: Results & Comparison

- Summarize results (table/plot):
 - Baseline (clean) vs. Augmented (if attempted)
 - Clean vs. each corrupted test set (Task 2.2)
- Brief discussion:
 - Which corruptions hurt most, and why?
 - Did augmentation help? Which transforms mattered?
 - What would you try next (e.g., stronger augmentation, adversarial training, denoising pre-processing, larger models, early stopping, ensembling)?

Model / Data Type	Accuracy	F1
Baseline	0.936	0.934
Augmented	0.835	0.797
Gaussian Noise	0.308	0.303
Motion Blur	0.406	0.414

The model did worst on the test set with Gaussian noise. This is because the noise changes many pixels randomly, so the model can't find the patterns it learned during training. Motion blur also lowered the accuracy, but not as much, since the shape of the objects is still there even if they are blurry. The model trained with data augmentation got lower validation accuracy than the original baseline and also overfitted more. Augmentation helped a bit, but not enough to match the baseline model. We would try using stronger augmentation (for example adding noise during training), early stopping to avoid overfitting.